Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации
Федеральное государственное бюджетное общеобразовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Практическая работа №4

по дисциплине
«Основы систем мобильной связи»
Тема: «Изучение корреляционных свойств последовательностей, используемых для синхронизации в сетях мобильной связи»

Выполнил:
студент гр. ИКС-433
Демин Сергей Алексеевич

Проверила:
Дроздова Вера Геннадьевна

Новосибирск, 2026

Содержание

1. Цель практической работы.....3
2. Задачи практической работы 3
3. Краткие теоретические сведения.....4
4. Исходные данные 6
5. Этапы выполнения работы.....6
6. Ответы на контрольные вопросы 18
7. Заключение.....19
8. Ссылка на Git 20

1. Цель практической работы

Получить представление о том, какие существуют псевдослучайные двоичные последовательности, какими корреляционными свойствами они обладают и как используются для синхронизации приемников и передатчиков в сетях мобильной связи.

2. Задачи практической работы

1) Напишите программу на языке C/C++ для генерации последовательности Голда, используя схему, изображенную на рисунке 4.4, если ваша группа с четным номером и 4.5 – если с нечетным, и порождающие полиномы *x* и *y*, при этом *x* – это ваш порядковый номер в журнале в двоичном формате (5 бит), а *y* – это *x+7* (5 бит). Например, ваш номер 22, значит:

x = 1 0 1 1 0, *y* = 1 1 1 0 1.

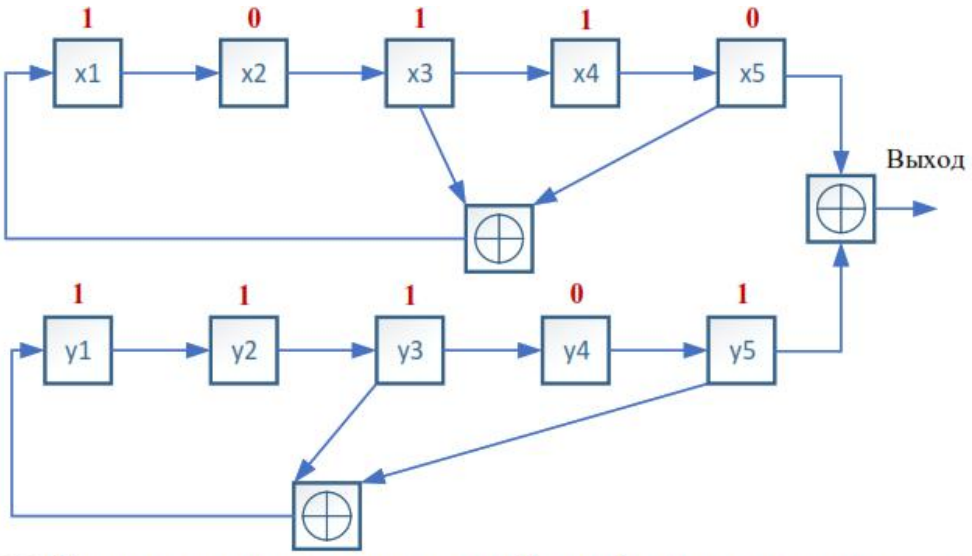


Рис. 4.4. Генерация последовательности Голда (вариант для *четного* номера группы)

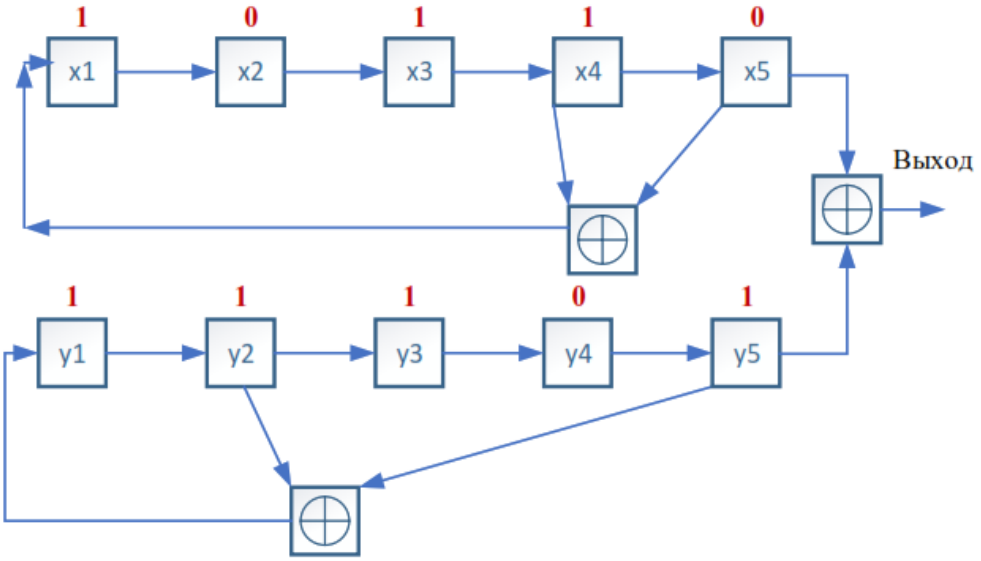


Рис. 4.5. Генерация последовательности Голда (вариант для *нечетного* номера группы)

2) Выведите получившуюся последовательность на экран.

3) Сделайте поэлементный циклический сдвиг последовательности и посчитайте автокорреляцию исходной последовательности и сдвинутой. Сформируйте таблицу с битовыми значениями последовательностей, в последнем столбце которой будет вычисленное значение автокорреляции, как показано в примере ниже.

Сдвиг	бит 1	бит 2	бит 3	Автокорреляция
0	1	1	0	1
1	0	1	1	-1/3
2	0	1	1	-1/3
3	1	1	0	1

4) Сформируйте еще одну последовательность Голда, используя свою схему (рис.4.4 или 4.5), такую что $x=x+1$, а $y= y-5$.

5) Вычислите значение взаимной корреляции исходной и новой последовательностей и выведите в терминал.

6) Проделайте шаги 1-5 в Matlab. Используйте функции xcorr() и autocorr() для вычисления соответствующих корреляций. Сравните результаты, полученные в Matlab и C/C++.

7) Выведите на график в Matlab функцию автокорреляции в зависимости от величины задержки (lag).

8) Составьте отчет. Отчет должен содержать титульный лист, содержание, цель и задачи работы, теоретические сведения, исходные данные, этапы выполнения работы, сопровождаемые скриншотами и графиками, демонстрирующими успешность выполнения, и промежуточными выводами, результирующими таблицами, ответы на контрольные вопросы, и заключение и **ссылка в виде QR-кода на репозиторий с кодом (git)**.

3. Краткие теоретические сведения

Псевдослучайные двоичные последовательности (PN-sequences – Pseudo-Noise) – это частный случай псевдослучайных последовательностей, элементами которой являются только 2 возможных значения (1 и 0 или -1 и +1). Такие последовательности очень часто используются в сетях мобильной связи. Возможные области применения:

- оценка вероятности битовой ошибки (BER – Bit Error Rate). В этом случае передатчик передает приемнику заранее известную PN-последовательность бит, а приемник анализируя значения бит на

конкретных позициях, вычисляет количество искаженных бит и вероятность битовой ошибки в текущих радиоусловиях, что затем может быть использовано для работы алгоритмов, обеспечивающих помехозащищенность системы;

- временная синхронизация между приемником и передатчиком. Включаясь абонентский терминал начинает записывать сигнал, дискретизируя его с требуемой частотой, в результате чего формируется массив временных отсчетов и требуется понять, начиная с какого элемента в этом массиве собственно содержатся какие-либо данные, как именно структурирована ось времени, где начинаются временные слоты. Используя заранее известную синхронизирующую PN-последовательность (синхросигнал), приемник сравнивает полученный сигнал с этой последовательностью на предмет «сходства» - корреляции. И если фиксируется корреляционный пик, то на стороне приема можно корректно разметить буфер с отсчетами на символы, слоты, кадры и пр.

- расширение спектра. Используется для повышения эффективности передачи информации с помощью модулированных сигналов через канал с сильными линейными искажениями (замираниями), делая систему устойчивой к узкополосным помехам (например, в 3G WCDMA).

Псевдослучайная битовая последовательность должна обладать следующими свойствами, чтобы казаться почти случайной:

1) Сбалансированность (balance), то есть число единиц и число нулей на любом интервале последовательности должно отличаться не более чем на одну.

2) Цикличность. Циклом в данном случае является последовательность бит с одинаковыми значениями. В каждом фрагменте псевдослучайной битовой последовательности примерно половину составляли циклы длиной 1, одну четверть – длиной 2, одну восьмую – длиной 3 и т.д.

3) Корреляция. Корреляция оригинальной битовой последовательности с ее сдвинутой копией должна быть минимальной. Автокорреляция этих последовательностей – это практически дельта-функция во временной области, как для аддитивного белого гауссовский шума AWGN (Additive white Gaussian noise), а в частотной области – это константа.

Чем длиннее последовательность, тем выше пик ее автокорреляционной функции, и тем больше напоминает дельта-функцию. Такого типа автокорреляцией характеризуется и белый

гауссовский шум, поэтому в англоязычной литературе такие последовательности называют pseudo noise sequences.

Чем острее автокорреляционный пик (то есть чем длиннее последовательность), тем удобней использовать данные последовательности для решения проблем синхронизации в сетях мобильной связи. Действительно, абонентский терминал при начальном включении должен засинхронизировать начало своих временных слотов на временной оси приемника и передатчика. Поэтому обычно базовые станции периодически отправляют специальные синхронизирующие последовательности, в качестве которых часто используются именно m-последовательности, и терминал вычисляет автокорреляцию этой заранее известной последовательности с полученным записанным сигналом, и в тот момент, когда фиксируется автокорреляционный пик, абонент отмечает начало слота на своей оси времени (а точнее номер отсчета в буфере, начиная с которого идет передаваемый базовой станцией слот с данными).

Разновидности псевдо-шумовых битовых последовательностей

М-последовательности – не единственные PN-последовательности, используемые в системах мобильной связи. Существуют также коды Баркера, коды Голда, коды Касами, коды Уолша-Адамара.

Коды Голда формируются путем суммирования по модулю 2 двух М-последовательностей одинаковой длины. Коды Касами также формируются из М-последовательностей путем взятия периодических выборок из этих последовательностей и суммированием их по модулю два. Данные коды обладают очень хорошими взаимокорреляционными свойствами.

4. Исходные данные

Группа ИКС-433 - номер нечетный
Номер в группе - 8.
X= 0 1 0 0 0
Y= 0 1 1 1 1

5. Этапы выполнения работы

1) Написал программу на языке C/C++ для генерации последовательности Голда, используя схему, изображенную на рисунке 4.5, так как моя группа с нечетным номером, и порождающие полиномы x и y, при этом x – это мой порядковый номер в журнале в двоичном формате (5 бит), а y – это x+7 (5 бит).

Мой номер 8, значит:

x = 0 1 0 0 0

y = 0 1 1 1 1

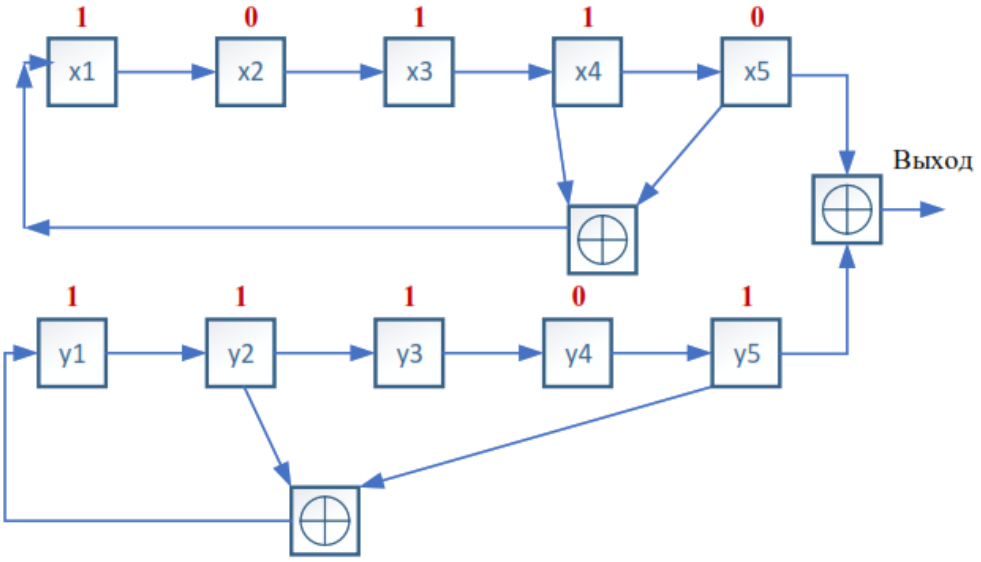


Рис. 4.5. Генерация последовательности Голда (вариант для *нечетного* номера группы)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int x[5]={0,1,0,0,0};
    int y[5]={0,1,1,1,1};

    int length=10;
    int* mas=(int*)malloc(length*sizeof(int));
    printf(" _____________________________________\n");
    printf("| Шаг |    X      |    Y      | Выход |\n");
    printf("|_____|___________|___________|_______|\n");

    for (int i=1;i<=length;i++){
        int out_x=x[4];
        int out_y=y[4];
        int result=out_x^out_y;

        mas[i-1]=result;

        printf("| %2d  |  ", i);

        for (int i=0;i<5;i++){
            printf("%d",x[i]);}

        printf("   |  ");

        for (int i=0;i<5;i++){
            printf("%d",y[i]);}

        printf("   |   %d   |\n",result);
```

```
        int new_bit_x=x[3]^x[4];
        int new_bit_y=y[1]^y[4];

        for (int i=4;i>0;i--){
            x[i]=x[i-1];
            y[i]=y[i-1];}

        x[0]=new_bit_x;
        y[0]=new_bit_y;}
    printf("|_____|_________|_________|_______|\n");

    printf("\nИтоговая последовательность: ");
    for (int i=0;i<length;i++){
        printf("%d",mas[i]);}
    printf("\n");

    free(mas);
}
```

2) Вывел получившуюся последовательность на экран.

```
segoga@NoteBook:/mnt/c/Users/sergx/OneDrive/Desktop/OCMC$ ./main
 _____________________________
| Шаг |    X    |    Y    | Выход |
|_____|_________|_________|_______|
|  1  |  01000  |  01111  |   1   |
|  2  |  00100  |  00111  |   1   |
|  3  |  00010  |  10011  |   1   |
|  4  |  10001  |  11001  |   0   |
|  5  |  11000  |  01100  |   0   |
|  6  |  01100  |  10110  |   0   |
|  7  |  00110  |  01011  |   1   |
|  8  |  10011  |  00101  |   0   |
|  9  |  01001  |  10010  |   1   |
| 10  |  10100  |  01001  |   1   |
|_____|_________|_________|_______|

Итоговая последовательность: 1110001011
```

3) Сделал поэлементный циклический сдвиг последовательности и посчитал автокорреляцию исходной последовательности и сдвинутой. Сформировал таблицу с битовыми значениями последовательностей, в последнем столбце которой вычисленное значение автокорреляции.

```
    int* shift=(int*)malloc(length*sizeof(int));

    printf(" _______");
    for (int i=0;i<length;i++){printf("_______");}
    printf("__________________\n");
```

```
    printf("| Сдвиг |");
    for (int i=1;i<=length;i++){printf(" бит%-2d|",i);}
    printf(" Автокорреляция |\n");

    printf("|_______|");
    for (int i=0;i<length;i++){printf("______|");}
    printf("__________________|\n");

    for (int s=0;s<length+1;s++){
        for (int j=0;j<length;j++){
            shift[j]=mas[(j+s)%length];}

        int sum=0;
        for (int j=0;j<length;j++){
            if (mas[j]==shift[j]){sum+=1;}
            else {sum-=1;}}
        double R=(double)sum/length;

        printf("|  %3d  |",s);
        for (int j=0;j<length;j++){printf("   %d  |",shift[j]);}
        printf("  %5.2f (%2d/%2d) |\n",R,sum,length); }

    printf("|_______|");
    for (int i=0;i<length;i++){printf("______|");}
    printf("__________________|\n");

    free(shift);
    free(mas);
}
```

```
 _____________________________________________________________________________________________
| Сдвиг | бит1 | бит2 | бит3 | бит4 | бит5 | бит6 | бит7 | бит8 | бит9 | бит10| Автокорреляция |
|_______|______|______|______|______|______|______|______|______|______|______|________________|
|   0   |   1  |   1  |   1  |   0  |   0  |   0  |   1  |   0  |   1  |   1  |  1.00 (10/10)  |
|   1   |   1  |   1  |   0  |   0  |   0  |   1  |   0  |   1  |   1  |   1  |  0.20 ( 2/10)  |
|   2   |   1  |   0  |   0  |   0  |   1  |   0  |   1  |   1  |   1  |   1  |  0.20 ( 2/10)  |
|   3   |   0  |   0  |   0  |   1  |   0  |   1  |   1  |   1  |   1  |   1  | -0.20 (-2/10)  |
|   4   |   0  |   0  |   1  |   0  |   1  |   1  |   1  |   1  |   1  |   0  | -0.20 (-2/10)  |
|   5   |   0  |   1  |   0  |   1  |   1  |   1  |   1  |   1  |   0  |   0  | -0.60 (-6/10)  |
|   6   |   1  |   0  |   1  |   1  |   1  |   1  |   1  |   0  |   0  |   0  | -0.20 (-2/10)  |
|   7   |   0  |   1  |   1  |   1  |   1  |   1  |   0  |   0  |   0  |   1  | -0.20 (-2/10)  |
|   8   |   1  |   1  |   1  |   1  |   1  |   0  |   0  |   0  |   1  |   0  |  0.20 ( 2/10)  |
|   9   |   1  |   1  |   1  |   1  |   0  |   0  |   0  |   1  |   0  |   1  |  0.20 ( 2/10)  |
|  10   |   1  |   1  |   1  |   0  |   0  |   0  |   1  |   0  |   1  |   1  |  1.00 (10/10)  |
|_______|______|______|______|______|______|______|______|______|______|______|________________|
segoga@NoteBook:/mnt/c/Users/sergx/OneDrive/Desktop/OCMC$
```

4) Сформировал еще одну последовательность Голда, используя свою схему, такую что x=x+1, а y= y-5.

```
#include <stdio.h>
#include <stdlib.h>
```

```
void gen_sequence(int x[5], int y[5], int length, int* mas){
    printf(" _____________________________________\n");
    printf("| Шаг |    X     |    Y     | Выход |\n");
    printf("|_____|__________|__________|_______|\n");

    for (int i=1;i<=length;i++){
        int out_x=x[4];
        int out_y=y[4];
        int result=out_x^out_y;

        mas[i-1]=result;

        printf("| %2d  |  ", i);

        for (int i=0;i<5;i++){
            printf("%d",x[i]);}

        printf("   |  ");

        for (int i=0;i<5;i++){
            printf("%d",y[i]);}

        printf("   |   %d   |\n",result);


        int new_bit_x=x[3]^x[4];
        int new_bit_y=y[1]^y[4];

        for (int i=4;i>0;i--){
            x[i]=x[i-1];
            y[i]=y[i-1];}

        x[0]=new_bit_x;
        y[0]=new_bit_y;}
    printf("|_____|__________|__________|_______|\n");
}


int main() {
    int x[5]={0,1,0,0,0};
    int y[5]={0,1,1,1,1};
    int x2[5]={0,1,0,0,1};
    int y2[5]={0,1,0,1,0};

    int length=10;
    int* mas=(int*)malloc(length*sizeof(int));\
    int* mas2=(int*)malloc(length*sizeof(int));
```

```
    printf("Первая последовательность\n");
    gen_sequence(x,y,length,mas);

    printf("\nПоследовательность: ");
    for (int i=0;i<length;i++){
        printf("%d",mas[i]);}
    printf("\n\n\n");

    printf("Вторая последовательность\n");
    gen_sequence(x2,y2,length,mas2);

    printf("\nПоследовательность: ");
    for (int i=0;i<length;i++){
        printf("%d",mas2[i]);}
    printf("\n");
}
```

```
Вторая последовательность
 _____________________________________
| Шаг |    X     |    Y     | Выход |
|_____|__________|__________|_______|
|  1  |  01001   |  01010   |   1   |
|  2  |  10100   |  10101   |   1   |
|  3  |  01010   |  11010   |   0   |
|  4  |  10101   |  11101   |   0   |
|  5  |  11010   |  01110   |   0   |
|  6  |  11101   |  10111   |   0   |
|  7  |  11110   |  11011   |   1   |
|  8  |  11111   |  01101   |   0   |
|  9  |  01111   |  00110   |   1   |
| 10  |  00111   |  00011   |   0   |
|_____|__________|__________|_______|

Последовательность: 1100001010
```

5) Вычислил значение взаимной корреляции исходной и новой последовательностей и вывел в терминал.

```
#include <stdio.h>
#include <stdlib.h>

void correlation(int* a, int* b, int length){

    int* shift=(int*)malloc(length*sizeof(int));

    printf(" _______");
    for (int i=0;i<length;i++){printf("_______");}
```

```
    printf("_________________\n");

    printf("| Сдвиг |");
    for (int i=1;i<=length;i++){printf(" бит%-2d|",i);}
    printf(" Автокорреляция |\n");

    printf("|_______|");
    for (int i=0;i<length;i++){printf("______|");}
    printf("_________________|\n");

    for (int s=0;s<length+1;s++){
        for (int j=0;j<length;j++){
            shift[j]=b[(j+s)%length];}

        int sum=0;
        for (int j=0;j<length;j++){
            if (a[j]==shift[j]){sum+=1;}
            else {sum-=1;}}

        double R=(double)sum/length;

        printf("|  %3d  |",s);
        for (int j=0;j<length;j++){printf("   %d  |",shift[j]);}
        printf("  %5.2f (%2d/%2d) |\n",R,sum,length); }

    printf("|_______|");
    for (int i=0;i<length;i++){printf("______|");}
    printf("_________________|\n");
    free(shift);

}


int main() {
    int x[5]={0,1,0,0,0};
    int y[5]={0,1,1,1,1};
    int x2[5]={0,1,0,0,1};
    int y2[5]={0,1,0,1,0};

    int length=10;
    int* mas=(int*)malloc(length*sizeof(int));
    int* mas2=(int*)malloc(length*sizeof(int));


    printf("Первая последовательность\n");
    gen_sequence(x,y,length,mas);
```

```
    printf("\nПоследовательность: ");
    for (int i=0;i<length;i++){
        printf("%d",mas[i]);}
    printf("\n\n\n");

    printf("Вторая последовательность\n");
    gen_sequence(x2,y2,length,mas2);

    printf("\nПоследовательность: ");
    for (int i=0;i<length;i++){
        printf("%d",mas2[i]);}
    printf("\n");

    printf("\n\nАвтокорреляция первой:\n");
    correlation(mas, mas, length);

    printf("\n\nВзаимная корреляция первой от второй:\n");
    correlation(mas, mas2, length);

    printf("\n\nВзаимная корреляция второй от первой:\n");
    correlation(mas2, mas, length);

    free(mas);
}
```

```
Взаимная корреляция первой от второй:
 _____________________________________________________________________________
| Сдвиг | бит1 | бит2 | бит3 | бит4 | бит5 | бит6 | бит7 | бит8 | бит9 | бит10| Автокорреляция |
|_______|______|______|______|______|______|______|______|______|______|______|________________|
|    0  |   1  |   1  |   0  |   0  |   0  |   0  |   1  |   0  |   1  |   0  |   0.60 ( 6/10) |
|    1  |   1  |   0  |   0  |   0  |   0  |   1  |   0  |   1  |   0  |   1  |  -0.20 (-2/10) |
|    2  |   0  |   0  |   0  |   0  |   1  |   0  |   1  |   0  |   1  |   1  |   0.20 ( 2/10) |
|    3  |   0  |   0  |   0  |   1  |   0  |   1  |   0  |   1  |   1  |   0  |  -0.60 (-6/10) |
|    4  |   0  |   0  |   1  |   0  |   1  |   0  |   1  |   1  |   0  |   0  |  -0.20 (-2/10) |
|    5  |   0  |   1  |   0  |   1  |   0  |   1  |   1  |   0  |   0  |   0  |  -0.20 (-2/10) |
|    6  |   1  |   0  |   1  |   0  |   1  |   1  |   0  |   0  |   0  |   0  |  -0.20 (-2/10) |
|    7  |   0  |   1  |   0  |   1  |   1  |   0  |   0  |   0  |   0  |   1  |  -0.20 (-2/10) |
|    8  |   1  |   0  |   1  |   1  |   0  |   0  |   0  |   0  |   1  |   0  |   0.20 ( 2/10) |
|    9  |   0  |   1  |   1  |   0  |   0  |   0  |   0  |   1  |   0  |   1  |   0.20 ( 2/10) |
|   10  |   1  |   1  |   0  |   0  |   0  |   0  |   1  |   0  |   1  |   0  |   0.60 ( 6/10) |
|_______|______|______|______|______|______|______|______|______|______|______|________________|


Взаимная корреляция второй от первой:
 _____________________________________________________________________________
| Сдвиг | бит1 | бит2 | бит3 | бит4 | бит5 | бит6 | бит7 | бит8 | бит9 | бит10| Автокорреляция |
|_______|______|______|______|______|______|______|______|______|______|______|________________|
|    0  |   1  |   1  |   1  |   0  |   0  |   0  |   1  |   0  |   1  |   1  |   0.60 ( 6/10) |
|    1  |   1  |   1  |   0  |   0  |   0  |   1  |   0  |   1  |   1  |   1  |   0.20 ( 2/10) |
|    2  |   1  |   0  |   0  |   0  |   1  |   0  |   1  |   1  |   1  |   1  |   0.20 ( 2/10) |
|    3  |   0  |   0  |   0  |   1  |   0  |   1  |   1  |   1  |   1  |   1  |  -0.20 (-2/10) |
|    4  |   0  |   0  |   1  |   0  |   1  |   1  |   1  |   1  |   1  |   0  |  -0.20 (-2/10) |
|    5  |   0  |   1  |   0  |   1  |   1  |   1  |   1  |   1  |   0  |   0  |  -0.20 (-2/10) |
|    6  |   1  |   0  |   1  |   1  |   1  |   1  |   1  |   0  |   0  |   0  |  -0.20 (-2/10) |
|    7  |   0  |   1  |   1  |   1  |   1  |   1  |   0  |   0  |   0  |   1  |  -0.60 (-6/10) |
|    8  |   1  |   1  |   1  |   1  |   1  |   0  |   0  |   0  |   1  |   0  |   0.20 ( 2/10) |
|    9  |   1  |   1  |   1  |   1  |   0  |   0  |   0  |   1  |   0  |   1  |  -0.20 (-2/10) |
|   10  |   1  |   1  |   1  |   0  |   0  |   0  |   1  |   0  |   1  |   1  |   0.60 ( 6/10) |
|_______|______|______|______|______|______|______|______|______|______|______|________________|
segoga@NoteBook:/mnt/c/Users/sergx/OneDrive/Desktop/OCMC$
```

6) Проделал шаги 1-5 в Matlab. Использовал функции xcorr() и autocorr() для вычисления соответствующих корреляций. Сравнил результаты, полученные в Matlab и C/C++.

```
function [mas,x,y]=gen_sequence(x,y,length)
    fprintf(" _____________________________________\n");
    fprintf("| Шаг |    X     |    Y     | Выход |\n");
    fprintf("|_____|__________|__________|_______|\n");

    mas=zeros(1,length);

    for i=1:length
        out_x=x(5);
        out_y=y(5);
        result=bitxor(out_x,out_y);

        mas(i)=result;

        fprintf("| %2d  |  ", i);
        fprintf("%d", x);
        fprintf("   |  ");
        fprintf("%d", y);
        fprintf("   |   %d   |\n", result);

        new_bit_x=bitxor(x(4),x(5));
        new_bit_y=bitxor(y(2),y(5));

        x(2:5)=x(1:4);
        y(2:5)=y(1:4);

        x(1)=new_bit_x;
        y(1)=new_bit_y;
    end
    fprintf("|_____|__________|__________|_______|\n");
end

length=10;

x =[0 1 0 0 0];
y =[0 1 1 1 1];
x2=[0 1 0 0 1];
y2=[0 1 0 1 0];

fprintf("Первая последовательность\n");
[mas,x,y]=gen_sequence(x,y,length);

fprintf("\nПоследовательность: ");
fprintf("%d", mas);
fprintf("\n\n\n");

fprintf("Вторая последовательность\n");
[mas2,x2,y2]=gen_sequence(x2,y2,length);
```

```
fprintf("\nПоследовательность: ");
fprintf("%d", mas2);
fprintf("\n");

fprintf("\n\nАвтокорреляция первой:\n");
[ac,lags]=autocorr(mas,'NumLags',length-1);
T=table(lags(:),ac(:),'VariableNames',{'Сдвиг','Автокорреляция'});
disp(T);


fprintf("\n\nВзаимная корреляция первой от второй:\n");
[c,lags]=xcorr(mas,mas2,'coeff');
idx=(lags>=0)&(lags<=length);
c_vals=c(idx);
lags_vals=lags(idx);
T=table(lags_vals(:),c_vals(:),'VariableNames',{'Сдвиг', 'Взаимная
корреляция'});
disp(T);


fprintf("\n\nВзаимная корреляция второй от первой:\n");
[c,lags]=xcorr(mas2,mas,'coeff');
idx=(lags>=0)&(lags<=length);
c_vals=c(idx);
lags_vals=lags(idx);
T=table(lags_vals(:),c_vals(:),'VariableNames',{'Сдвиг', 'Взаимная
корреляция'});
disp(T);
```

Автокорреляция
1.00 (10/10)
0.20 (2/10)
0.20 (2/10)
-0.20 (-2/10)
-0.20 (-2/10)
-0.60 (-6/10)
-0.20 (-2/10)
-0.20 (-2/10)
0.20 (2/10)
0.20 (2/10)
1.00 (10/10)

Автокорреляция первой:

Сдвиг	Автокорреляция
0	1
1	0.1
2	0.033333
3	-0.28333
4	-0.18333
5	-0.33333
6	-0.066667
7	0.033333
8	0.13333
9	0.066667

Автокорреляция
0.60 (6/10)
-0.20 (-2/10)
0.20 (2/10)
-0.60 (-6/10)
-0.20 (-2/10)
-0.20 (-2/10)
-0.20 (-2/10)
-0.20 (-2/10)
0.20 (2/10)
0.20 (2/10)
0.60 (6/10)

Взаимная корреляция первой от второй:

Сдвиг	Взаимная корреляция
0	0.8165
1	0.61237
2	0.40825
3	0.20412
4	-3.626e-17
5	0.20412
6	0.20412
7	0.20412
8	0.40825
9	0.20412

Автокорреляция
0.60 (6/10)
0.20 (2/10)
0.20 (2/10)
-0.20 (-2/10)
-0.20 (-2/10)
-0.20 (-2/10)
-0.20 (-2/10)
-0.60 (-6/10)
0.20 (2/10)
-0.20 (-2/10)
0.60 (6/10)

Взаимная корреляция второй от первой:

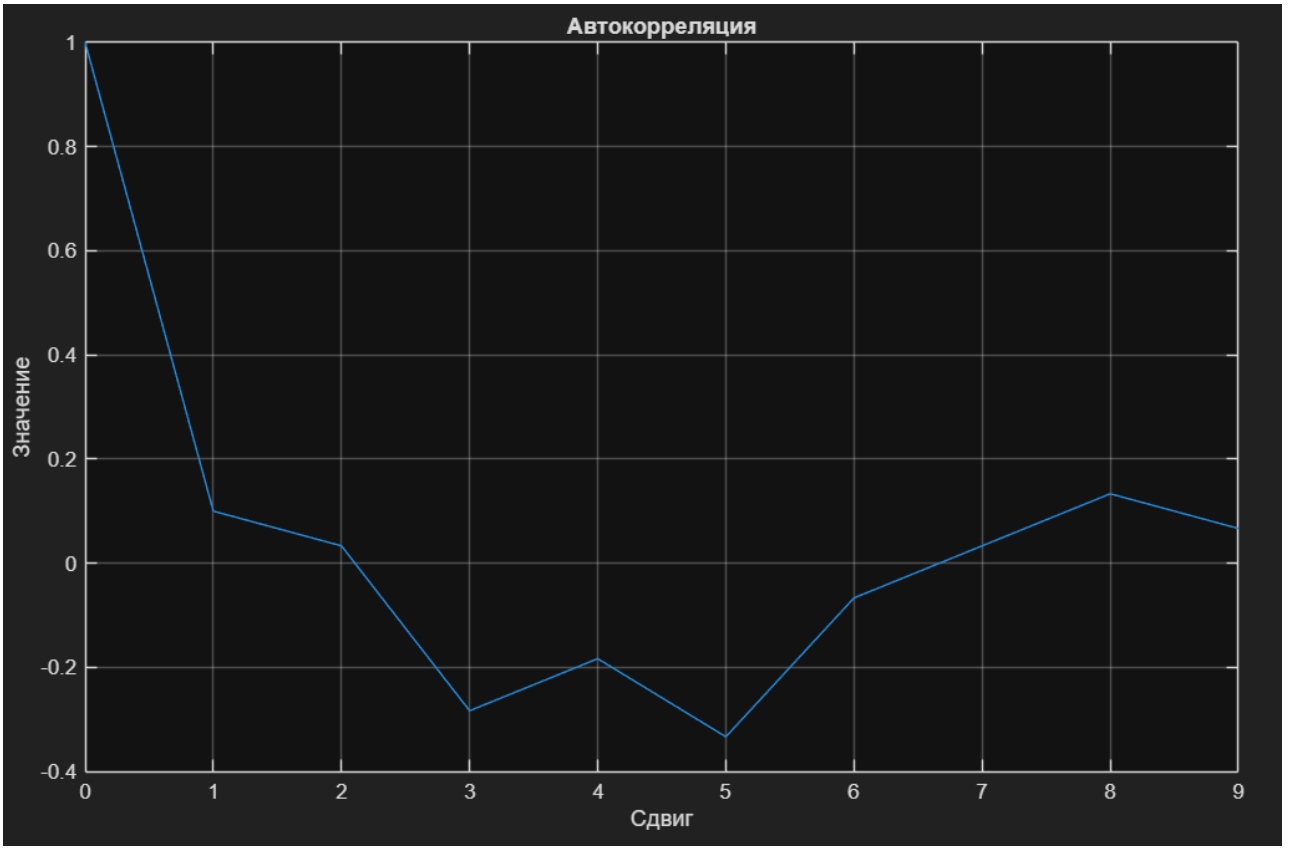
Сдвиг	Взаимная корреляция
0	0.8165
1	0.20412
2	0.20412
3	3.626e-17
4	0.20412
5	0.20412
6	0.40825
7	0.20412
8	0.20412
9	0

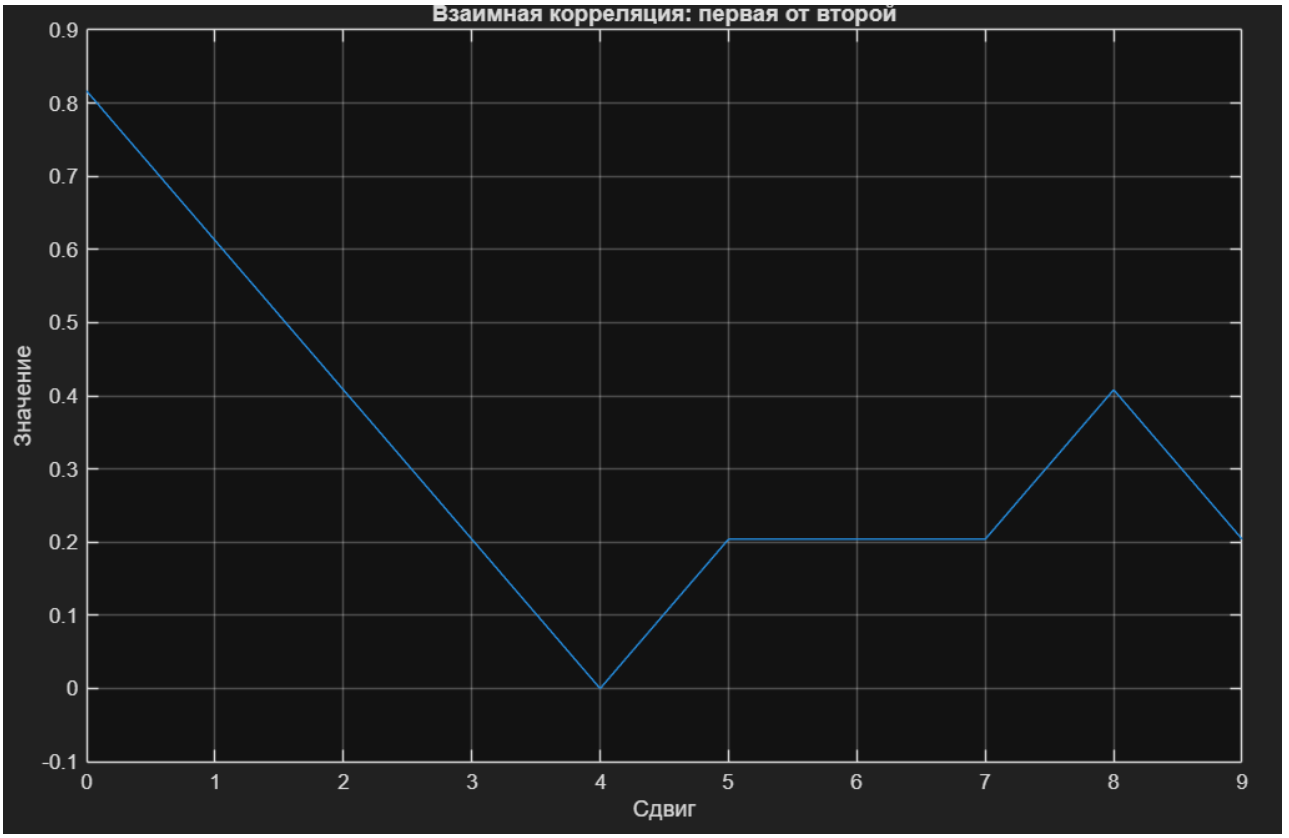
7) Вывел на график в Matlab функцию автокорреляции в зависимости от величины задержки (lag).

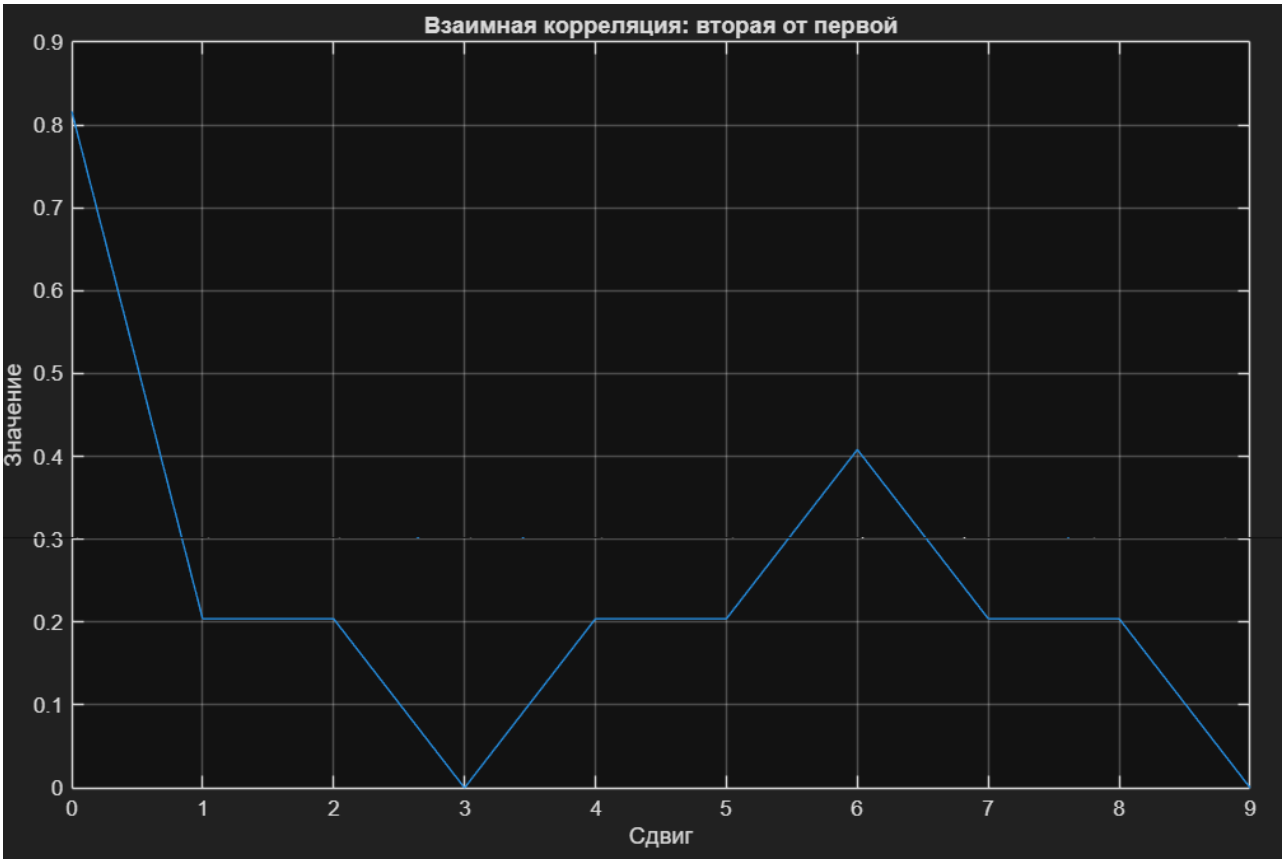
```
figure;
plot(lags,ac);
title('Автокорреляция');
xlabel('Сдвиг');
ylabel('Значение');
grid on;


figure;
plot(lags_vals,c_vals);
title('Взаимная корреляция: первая от второй');
xlabel('Сдвиг');
ylabel('Значение');
grid on;


figure;
plot(lags_vals,c_vals);
title('Взаимная корреляция: вторая от первой');
xlabel('Сдвиг');
ylabel('Значение');
grid on;
```


Автокорреляция
Значение
Сдвиг
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
0
1
2
3
4
5
6
7
8
9


Взаимная корреляция: первая от второй
Значение
Сдвиг
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
-0.1
0
1
2
3
4
5
6
7
8
9



6. Ответы на контрольные вопросы

1) Для чего в мобильных сетях могут использоваться псевдослучайные последовательности?

Они могут использоваться для:
1. Оценки вероятности битовой ошибки (передатчик отправляет известную последовательность, а приемник считает количество искаженных бит)
2. Временной синхронизации между приемником и передатчиком (приемник сравнивает принятый сигнал с известной синхропоследовательностью для нахождения корреляционного пика и разметки буфера на символы, слоты и кадры)
3. Расширения спектра (для повышения помехоустойчивости системы к узкополосным помехам)

2) Что значит положительная корреляция сигналов?

Положительная корреляция означает, что при сравнении двух последовательностей число совпадающих бит больше числа различающихся

3) Что такое корреляционный прием сигналов?

Это метод приема, при котором принятый сигнал перемножается и интегрируется с заранее известной опорной копией сигнала (синхропоследовательностью)

4) Как вычисление корреляционных функций помогает синхронизироваться приемникам и передатчика в сетях мобильной связи?

Базовая станция периодически отправляет синхронизирующую последовательность. Абонентский терминал вычисляет автокорреляцию этой известной последовательности с записанным сигналом. В момент, когда фиксируется корреляционный пик, терминал определяет начало слота на своей оси времени

5) Какими свойствами обладают псевдослучайные битовые последовательности?

1. Сбалансированность (число 1 и 0 на любом интервале должно отличаться не более чем на одну)
2. Цикличность (в последовательности примерно половину состовляют циклы длиной 1, четверть - длиной 2, восьмую - длиной 3 и тд.)
3. Корреляция (автокорреляция оригинальной последовательности с ее сдвинутой копией должна быть минимальной, а при полном совпадении - максимальной)

6) Какие разновидности PN-последовательностей вам известны?

1. М-последовательности
2. Коды Баркера
3. Коды Голда
4. Кода Касами
5. Коды Уошла-Адамара

7. Заключение

В ходе практической работы были изучены псевдослучайные двоичные последовательности и их корреляционные свойства. В C/C++ и Matlab были сгенерированы две последовательности Голда, рассчитаны их автокорреляция и взаимная корреляция, построены графики зависимости корреляции от задержки. Результаты в обеих средах совпали, что подтверждает корректность алгоритмов. Анализ показал, что автокорреляционная функция имеет острый пик при нулевом сдвиге, а взаимная корреляция остаётся низкой - именно эти свойства обеспечивают возможность синхронизации приёмника и передатчика в сетях мобильной связи. Цель работы достигнута, все задачи выполнены.

8. Ссылка на Git

